Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL604**	**Course Name:**	**Machine Learning Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**03**
Title of the Experiment:	**Implementation of Logistic Regression Algorithm.**
Date of Performance:	**29/01/2026**
Date of Submission:	**05/02/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Mrs. Sneha Yadav

Signature :

Date:

Aim: Implementation of Logistic Regression Algorithm.

Objective: Able to perform various feature engineering tasks, apply logistic regression on the given dataset and maximize the accuracy.

Theory:

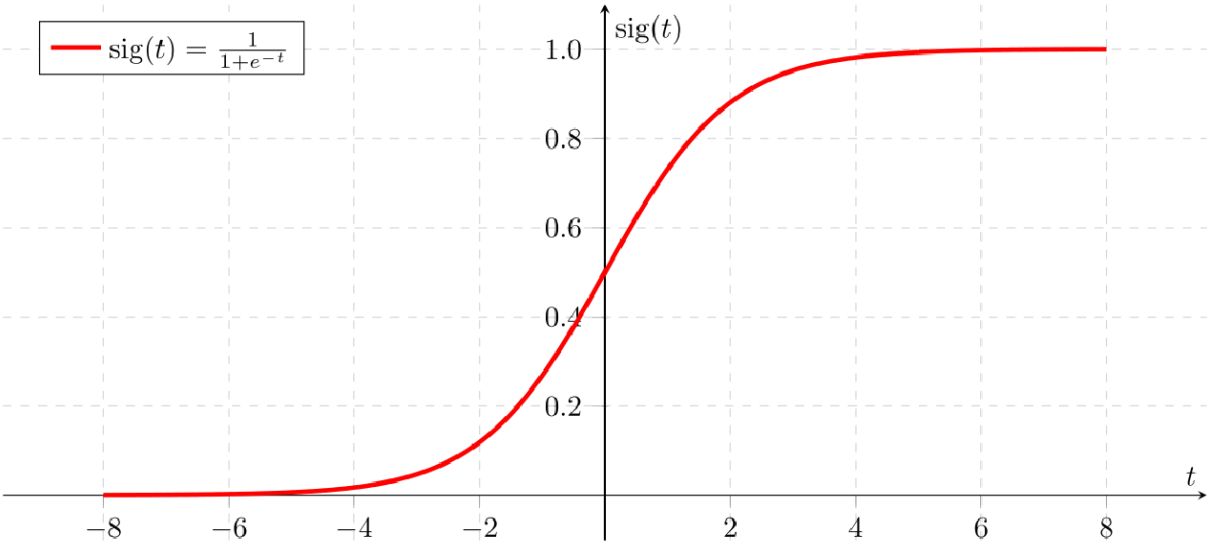
Logistic Regression was used in the biological sciences in early twentieth century. It was then used in many social science applications. Logistic Regression is used when the dependent variable(target) is categorical and is binary in nature. In order to perform binary classification, the logistic regression techniques makes use of Sigmoid function.

For example,

To predict whether an email is spam (1) or (0)

Whether the tumor is malignant (1) or not (0)

Consider a scenario where we need to classify whether an email is spam or not. If we use linear regression for this problem, there is a need for setting up a threshold based on which classification can be done. Say if the actual class is malignant, predicted continuous value 0.4 and the threshold value is 0.5, the data point will be classified as not malignant which can lead to serious consequence in real time.



From this example, it can be inferred that linear regression is not suitable for classification problem. Linear regression is unbounded, and this brings logistic regression into picture. Their value strictly ranges from 0 to 1.

Implementation:

1 . Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
!pip install scikit-learn
```

2: Load Dataset

```
col_names = ['pregnant', 'glucose', 'bp', 'skin',
             'insulin', 'bmi', 'pedigree', 'age', 'label']

pima = pd.read_csv("diabetes.csv",
                   header=None,
                   names=col_names)

pima.head()
```

	pregnant	glucose	bp	skin	insulin	bmi	pedigree	age	label
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

3: Feature Selection

```
feature_cols = ['pregnant', 'insulin', 'bmi', 'age',
                'glucose', 'bp', 'pedigree']

X = pima[feature_cols]   # Independent variables
y = pima.label           # Dependent variable
```

4: Train-Test Split

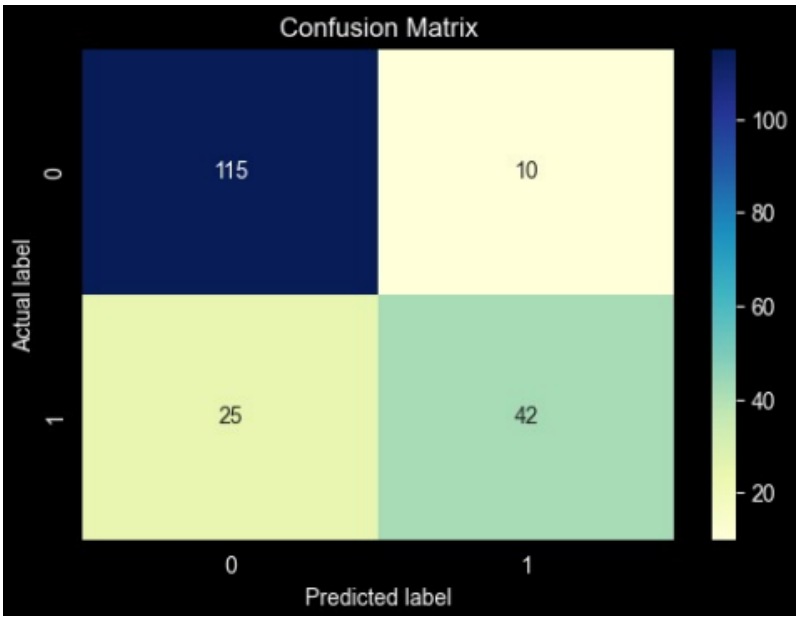
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=16
)
```

5: Model Building

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(random_state=16, max_iter=1000)
logreg.fit(X_train, y_train)
LogisticRegression(max_iter=1000, random_state=16)
```

6: Prediction

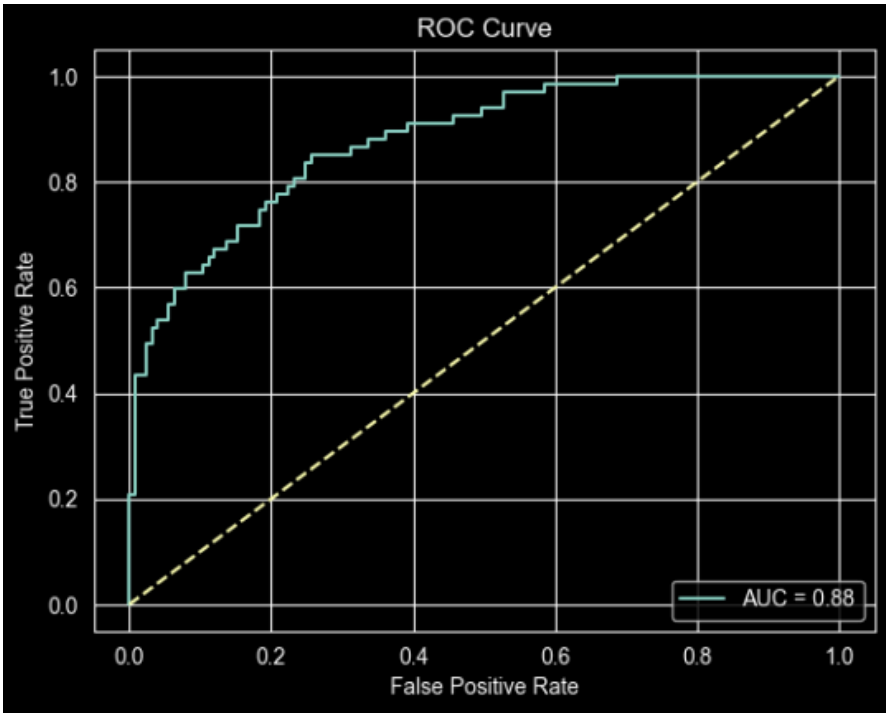
```
y_pred = logreg.predict(X_test)
from sklearn import metrics

cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
cnf_matrix
array([[115,  10],
       [ 25,  42]])
class_names = [0, 1]

plt.figure(figsize=(6,4))
sns.heatmap(pd.DataFrame(cnf_matrix),
            annot=True,
            cmap="YlGnBu",
            fmt='g')

plt.title("Confusion Matrix")
plt.ylabel("Actual label")
plt.xlabel("Predicted label")
plt.show()
```



```
from sklearn.metrics import classification_report

target_names = ['without diabetes', 'with diabetes']
print(classification_report(y_test, y_pred, target_names=target_names))
```

```
                   precision    recall  f1-score   support

without diabetes       0.82      0.92      0.87       125
   with diabetes       0.81      0.63      0.71        67

        accuracy                           0.82       192
       macro avg       0.81      0.77      0.79       192
    weighted avg       0.82      0.82      0.81       192
```

```
y_pred_proba = logreg.predict_proba(X_test)[:,1]

fpr, tpr, _ = metrics.roc_curve(y_test, y_pred_proba)
auc = metrics.roc_auc_score(y_test, y_pred_proba)

plt.plot(fpr, tpr, label="AUC = " + str(round(auc, 2)))
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend(loc=4)
plt.show()
```



Conclusion:

The logistic regression model achieves an **overall accuracy of 82%**, which indicates good performance on the test data. This means the model correctly classifies about **4 out of 5 instances**.

Looking beyond accuracy:

- For **patients without diabetes**, the model performs very well, with a **high recall of 0.92**, meaning most non-diabetic cases are correctly identified.
- For **patients with diabetes**, the **recall drops to 0.63**, indicating that some diabetic cases are missed (false negatives), which is a concern in medical diagnosis.
- Precision for both classes is fairly balanced (~0.81–0.82), showing the predictions are reasonably reliable.
- The **F1-score** is higher for the non-diabetic class (0.87) than the diabetic class (0.71), reflecting this imbalance in performance.

While the model demonstrates **good overall accuracy**, it is **better at identifying non-diabetic patients than diabetic ones**. In a healthcare context, improving recall for the diabetic class would be important, even if it slightly reduces overall accuracy.